

DECK 01 • WEEK 1

Foundations. Solana & learning with AI.

No coding this week. Build the mental models
you'll need before you touch a terminal.

ACCOUNTS • PROGRAMS • PDAs
TOKENS • DEV ENVIRONMENT

SECTION 01

How to learn with AI.

USING AI WELL TAKES PRACTICE

There's a difference between using AI to skip past things you don't understand and using AI to understand them faster.

Every week after this one involves hands-on building, and AI tools will be part of your workflow. Having access to a chatbot isn't the same as knowing how to use it.

In this section we'll cover four habits: **the verify loop**, asking "why" not "how", **building mental models**, and **knowing what AI gets wrong about Solana specifically.**

THE VERIFY LOOP

AI explanations can be wrong in subtle ways. Build the loop into your reading.

- 01 **Ask AI to explain a concept.** Set the level — beginner, “as if I know Ethereum”, whatever fits.
- 02 **Restate it in your own words.** Out loud, in writing, or by typing it back to AI. The teaching test.
- 03 **Check against the official docs** or a known-good source. Look for what AI omitted or smoothed over.
- 04 **If there’s a gap, ask AI about the discrepancy.** Don’t just accept the first answer. Probe.



SYCOPHANCY WARNING

AI sycophancy is when a model prioritises agreeing with you over giving accurate information. If AI changes its answer the moment you push back, that’s a tell. Re-verify against docs, not the model.

This 4-step process takes longer than accepting the first AI answer, but **it helps the knowledge stick.**

ASK "WHY" MORE THAN "HOW"

HOW QUESTION

"How do I create a PDA?"

Gives you code to copy.

WHY QUESTION

"Why do PDAs exist?"

Gives you understanding you can apply.

THREE PROMPTS THAT BUILD MENTAL MODELS

"Explain Solana's account model as if accounts were files on a computer. Where does the analogy break down?"

"I understand Ethereum's storage model. What's the equivalent in Solana, and what's fundamentally different?"

"Walk me through what happens step by step when I send a Solana transaction, from click to confirmation."



THIS WEEK FOCUSES ON WHY

The coding weeks handle "how". If you skip the "why" you'll be able to follow instructions but you won't know what to do when things break.

A **mental model** is your internal map of how something works and why. Good "why" questions push past surface answers, reveal edge cases, and help you reason through new problems without copying code.

FOUR FAILURE MODES TO WATCH FOR

The model can sound confident and still be out of date. Your job is to detect mismatches and ground answers in current docs.

01 • STALE VS CURRENT

Version drift is normal.

AI may return `web3.js v1` syntax when you're on v2, or old Anchor patterns. Always pin your versions first, then compare AI output against those versions.

02 • KNOWLEDGE CUTOFF GAPS

New changes may be missing.

If a feature is very recent, the model may not know it yet. When answers feel vague or conflict with your setup, bring the relevant doc snippet into the prompt and ask AI to re-evaluate from that source.

03 • MISSING LOCAL CONTEXT

Correct idea, wrong for your project.

Snippets often assume imports, config, accounts, or crate versions you don't have. Treat failures as context mismatches first: list your setup, then ask for a version-compatible rewrite.

04 • SAFETY BLIND SPOTS

Secure-by-default is not guaranteed.

AI-generated Solana code often under-specifies account validation, signer checks, and authority constraints. Fine for learning drafts; **dangerous for real funds** unless you verify each check explicitly.

READ EVERY LINE BEFORE YOU RUN IT

When AI generates 40 lines of working code and you hit accept, it feels like progress. It isn't, if you can't explain what it does and why.

In any context, blindly accepting code is risky. **When you're learning, the cost is even higher:** you can complete exercises without being able to explain or debug what you built.

The habit: when AI generates code, read every line. Match it back. *"This line creates a PDA with these seeds. This line transfers via CPI. This constraint checks the signer."*



CATCH YOURSELF

It's easy to start mass-accepting suggestions when you're tired or stuck. If you're accepting without reading, you're not learning — you're just generating files.

THE AI RAMP — GUIDANCE STEPS BACK EACH WEEK

- 1** **Week 1 (this week):** Use AI to explore concepts. Practice the verify loop.
- 2** **Weeks 2–3:** AI helps you write and debug code. Exercises tell you what to watch out for.
- 4** **Weeks 4–5:** AI is a collaborator. Less guidance. You're developing your own workflow.
- 6** **Week 6:** AI is your project partner. You decide when and how.

A BLOCKCHAIN DESIGNED FOR SPEED

Transactions confirm in **under a second**. Cost: fractions of a cent.

That makes Solana practical for applications other chains can't support economically: **high-frequency trading, real-time games, consumer apps** where users won't pay \$5 per transaction.

For developers, the practical differences are in how data storage, program execution, and cost work. The rest of this deck unpacks each one.

THREE ARCHITECTURAL CHOICES

Parallel execution

Transactions touching different accounts run on different cores at the same time.

Proof of History

Orders transactions *before* consensus, cutting validator agreement time.

Single runtime (SVM)

One virtual machine, optimised for throughput.

SECTION 03 • CORE CONCEPT

The accounts model.

If you only understand one thing about
Solana before next week, this is it.

ALL DATA LIVES IN ACCOUNTS

An account is a chunk of storage with an address, an owner, a balance, and a data field.
The closest analogy: a file on a filesystem.

LOCATION

Address

Like a file path. A unique 32-byte public key that tells you where the account lives.

PERMISSIONS

Owner

Like file permissions. The owner is the program that's allowed to modify this account's data.

CONTENTS

Data

Like file contents. Arbitrary bytes the program structures however it wants.



VERIFY-LOOP PROMPT

"I think Solana accounts are like files where the program is the app that reads/writes them, and the owner field is like file permissions. Where does this analogy break down?"

EVERY ACCOUNT HAS THESE FIVE FIELDS

Five fields. Memorise the names — you’ll see them every week.

address	Pubkey	Unique 32-byte public key. How you find and identify the account.
owner	Pubkey	The program that controls this account. Only the owner program can modify the account’s data. This is Solana’s permission model.
lamports	u64	SOL balance, in the smallest denomination. <code>1 SOL = 1,000,000,000 lamports</code> .
data	Vec<u8>	A byte array that can store anything. The program defines the structure. The chain doesn’t know or care what’s in here — that’s the program’s job.
executable	bool	Whether this account contains a program (code) or just data.

PROGRAMS AND DATA ARE SEPARATE

A program is deployed once and is **stateless**. All the data it works with lives in separate accounts that get passed in per transaction.

ETHEREUM

A smart contract **stores its own data internally**. Code and storage live together. Two transactions touching the same contract must run sequentially because they share global state.

SOLANA

Programs and data are separate. If two transactions touch different accounts, Solana processes them at the same time on different CPU cores. **That separation is what enables parallel execution.**



PARALLELISM COMES FROM THE ACCOUNT MODEL

In Solana, parallel execution follows from how accounts are modeled and declared. Because validators need to know read/write access ahead of time, each transaction must specify the accounts it will touch.

FOUR TYPES YOU'LL ENCOUNTER

All accounts use the same five fields — they differ by who owns them.

01 • WALLET ACCOUNT

Owned by the System Program.

Holds SOL. Controlled by whoever has the private key. Yours, when you generate a keypair.

02 • DATA ACCOUNT

Owned by a specific program.

Stores application state: token balances, voting records, game state — whatever the program needs.

03 • PROGRAM ACCOUNT

Marked as executable.

Contains the compiled program code itself. Read-only after deployment.

04 • TOKEN ACCOUNT

A specific data account.

Owned by the SPL Token program. Stores how many tokens of a specific type a wallet holds.

STORING DATA COSTS MONEY

Every account must maintain a minimum SOL balance proportional to its data size. **It's called rent exemption.**

If an account's balance drops below the threshold, **it gets deleted**. In practice you always fund accounts to be rent-exempt when you create them.

\$

SECURITY DEPOSIT, NOT A FEE

The SOL is **refunded when the account is closed**. Think of it as a deposit, not a recurring charge.

Bigger accounts cost more to keep alive. That's a forcing function: **store as little as you need**, and close accounts you're done with to reclaim the deposit.

WK 2 • EX 1

Hello Solana. You'll create your first account, fund it to be rent-exempt, and inspect the five fields with the Solana CLI. →

WK 2 • EX 2

Account inspector. Pull a real on-chain account, decode each field, and verify the rent-exemption math against current cluster epoch values. →

SECTION 04

Programs & transactions.

What a program actually *does*, and what happens when you send a transaction.

A SOLANA PROGRAM IS A COMPILED, STATELESS BINARY

Equivalent to a smart contract on Ethereum, with one key difference: **programs don't store data.** They receive accounts as input, read and modify them, return.

Written in **Rust** (most common), C, or any language that compiles to BPF bytecode. In practice, almost everyone uses Rust with the **Anchor framework**, which handles boilerplate and adds safety checks.

FOUR PRE-BUILT PROGRAMS YOU'LL INTERACT WITH

System Program

Hard-coded into the runtime. The only program that can create accounts and move native SOL.

SPL Token

Standard, audited, on-chain. Logic for minting, burning, and transferring assets.

Token-2022

Newer token standard. Programmable fees, metadata, custom hooks.

Associated Token Account

Utility program — deterministic link from wallet to token account.

A TRANSACTION IS A BUNDLE OF INSTRUCTIONS

Signed by one or more wallets, submitted to the network, and **atomic** — either all instructions succeed or none do.

EACH INSTRUCTION SPECIFIES

- 01 **Which program** to call.
- 02 **Which accounts** to pass in — and whether each is readable, writable, or a signer.
- 03 **Instruction data** — arbitrary bytes that tell the program what to do.



COMPOSABILITY IN ONE TX

A single transaction can contain multiple instructions to **different programs**. This is how you compose operations.

Example: *“create an account, initialise it with my program, transfer tokens to it”* — all atomic, in one transaction. If the transfer fails, the account isn’t left half-created.

CLICK SEND → ON-CHAIN, IN UNDER 500MS

Five steps. Each one matters when you're debugging.

- 1 Client constructs & signs.** Builds the transaction, signs with your wallet, sends to an RPC node.
- 2 RPC forwards.** The RPC node forwards it to the current leader validator.
- 3 Leader processes.** For each instruction: loads accounts, calls the program, the program succeeds or returns an error.
- 4 Commit or rollback.** All instructions succeed → state changes commit. Any failure → everything rolls back.
- 5 Block + verification.** Transaction is included in a block, broadcast, and verified by other validators.

COMPUTE UNITS

Every transaction has a budget measured in **compute units (CU)**. Default: **200,000 CU** per instruction. Max: **1.4M CU** per transaction. Exceed it → transaction fails.

Request more (up to the max) by including a compute-budget instruction, and set a **priority fee** in CUs to skip the queue during congestion.

WK 2 · EX 2

Send your first transaction. Build, sign, submit, trace it through the lifecycle in the explorer. Then exceed the CU budget on purpose to see the failure. →

SECTION 05 · CORE CONCEPT

Program Derived Addresses.

An important foundation. Read this section slowly. Come back to it if you need to.

PROGRAMS DON'T HAVE PRIVATE KEYS

Programs need to own accounts to store data. But they're code, not users. So how do they sign?

They don't need to. **PDAs are addresses mathematically derived from a set of inputs (seeds) and a program ID.** They're guaranteed to have *no* corresponding private key — so no one can sign for them.

Instead, the runtime lets the program that derived the PDA act as the signer through a mechanism called `invoke_signed`.

HOW A PDA IS GENERATED

A PDA is the result of hashing together:

- 01 One or more seeds** — arbitrary byte arrays you choose.
- 02 The program ID** — the program deriving the PDA.
- 03 A bump seed** — a number that pushes the result *off* the elliptic curve, guaranteeing no private key exists.

`find_program_address` tries bump values from 255 down and returns the first off-curve address. **Deterministic — same inputs, same PDA, every time.**

SEEDS = STRUCTURED, FINDABLE STORAGE

Determinism is what makes this useful. If you know the seeds, you can compute the address from anywhere.

PER-USER COUNTER

One PDA per user

```
["counter", user_wallet]
```

Each user gets their own counter at a predictable address.

VOTING PROPOSAL

One PDA per proposal

```
["proposal", proposal_id]
```

Anyone can compute the address of proposal #7 without querying the chain.

ESCROW VAULT

One PDA per escrow

```
["vault", escrow_id]
```

The program can find and control the vault for any escrow it created.

?

ASK AI

"If I'm building a blog on Solana where users can create posts, what seeds would I use? What if users can leave comments? What happens if two different users use the same seeds — how do I prevent PDA collisions?"

WK 3 • EX 5

Counter program with PDAs. Build a per-user counter. You'll derive PDAs client-side, pass them into instructions, and use `invoke_signed` for the program to sign. →

ONE PROGRAM MANAGES ALL TOKENS

On Ethereum, each token is its own contract.
On Solana, you create a mint account
that the SPL Token program manages.

01 PER TOKEN

Mint account

Defines a token type. Stores total supply, number of decimals, and who has authority to mint more.

One per token — one for USDC, one for BONK, one for your custom token.

02 PER USER • PER TOKEN

Token account

Holds a specific user's balance of a specific token. Each one references the mint and the wallet that owns it.

If you hold 5 tokens, you have 5 token accounts.

03 CONVENTION

Associated Token Account

A token account at a deterministic address derived from **wallet + mint**.

Compute the ATA address and you know where any wallet's balance for any token lives — no query needed.

TWO TOKEN PROGRAMS, ONE ECOSYSTEM

SPL Token is the original. Token-2022 adds **optional features (extensions)** on top of the same primitives.

SPL TOKEN

The original. Handles minting, transferring, burning, account management. Battle-tested. **Used by most existing tokens.**

TOKEN-2022 • TOKEN EXTENSIONS

Adds optional features: metadata on the mint (no Metaplex needed), transfer fees, transfer hooks, non-transferable (soulbound) tokens, confidential transfers.

**EXTENSIONS ARE PERMANENT**

Extensions are configured at mint creation time and become permanently part of that mint's behaviour. **You can't add extensions after creation.**

WK 3 • EX 3

Standard SPL token. Create a mint, mint supply to your ATA, transfer between two wallets. Inspect every account along the way. →

WK 3 • EX 4

Token-2022 with extensions. Create a mint with transfer fees and metadata extensions. Verify fees collect on every transfer. →

MENTAL MODEL SHIFTS

For Solidity / Hardhat developers. Skip if you're new to blockchain — this section assumes Ethereum knowledge.

CONCEPT	ETHEREUM / EVM	SOLANA
Data storage	Contract stores its own state internally (mappings, variables).	Programs are stateless. Data lives in separate accounts passed in per transaction.
Tokens	Each token = a separate smart contract (ERC-20).	One program (SPL Token) manages all tokens. You create mint accounts, not contracts.
Cost model	Gas price × gas used. Costs \$1–\$50+ per transaction.	Base fee + priority fee. Typically < \$0.01.
Execution	Sequential. One transaction at a time.	Parallel. Non-conflicting transactions run simultaneously.
Language	Solidity (most common).	Rust with Anchor framework (most common).
Accounts	Implicit. Contracts access storage slots internally.	Explicit. Every account a transaction touches must be declared upfront.
Composability	Call any contract from any contract freely.	Cross-Program Invocation (CPI). Must pass accounts through the call chain.

YOUR TOOLCHAIN — INTRODUCED, NOT INSTALLED

Next week you’ll install everything.
This is the lay of the land so the
names aren’t unfamiliar when you do.

Solana CLI

Configure network, manage wallets, airdrop test SOL, deploy programs.

Anchor

Rust framework. Macros for serialisation, validation, security checks. Includes a test framework.

Rust & TypeScript

Rust for on-chain programs. TS + web3.js v2 for clients (note: v2 ≠ v1).

RPC nodes

Your gateway to the network. Free public devnet endpoints (rate-limited). Helius / QuickNode / Triton for production.

CODING PLACEHOLDER • DETAILED BRIEF

DEVNET

Week 2 • Exercise 1 — Hello Solana

OBJECTIVE	Set up your dev environment, generate a keypair, request devnet SOL, and verify the airdrop transaction on-chain.
BUILDS	A funded devnet wallet + a saved transaction signature you can re-look-up in the explorer.
CONCEPTS	Accounts (slides 09–14) • System Program (16) • Lifecycle (18) • Network tiers (this slide).
AI USAGE	Use AI to explain CLI errors. Don’t let it skip the airdrop step “for you” — type each command yourself.
VERIFY	<code>solana balance</code> > 0 SOL on devnet, and the airdrop tx opens cleanly on Solana Explorer.

DEVNET — your playground

testnet — validators

mainnet — real money

NO CODING THIS WEEK — DO THESE FOUR THINGS

Build the mental models. Practice the loop. Get a head start on Rust if you want.

- 01 Read this document fully.** Don't skim. Accounts and PDAs are the foundation for everything that follows.
- 02 Practice the AI verify loop.** Pick two concepts (e.g. PDAs and accounts). Get AI to explain. Restate. Check against docs. Note discrepancies.
- 03 If you've never used Rust, read the basics.** Rust Book chapters 1–6. Focus: structs, enums, pattern matching, error handling.
- 04 Install your tools (optional).** Rust, Solana CLI, Anchor — or wait for Week 2 Exercise 1, which walks through it in detail.

RESOURCES — OPTIONAL DEEPER READING

Solana Core Concepts

solana.com/docs/core — start with Accounts, Transactions, Programs.

Anchor Documentation

anchor-lang.com/docs — skim Getting Started + Core Concepts this week.

SPL Token + Token Extensions

spl.solana.com/token • solana.com/developers/guides/token-extensions/getting-started

The Rust Book

doc.rust-lang.org/book — chapters 1–6 are sufficient for this course.